

# NUMERICAL ANALYSIS

## GUIDED LAB 4

ROB TOMPKINS

Note, all the files associated with Guided Lab 4 are checked into the :

<https://github.com/chtompki/MathSpring2026/tree/main/NumericalAnalysis/GuidedLab4>

---

1. Given an initial value problem  $y' = f(t, y)$  for  $t \in [a, b]$  with initial condition  $y(a) = \alpha$ , create a program that implements RK4 to approximate the solution  $w_i \approx y(t_i)$  with fixed step-size  $h = (b - a)/N$ :

$$\begin{aligned}k_1 &= hf(t_i, w_i) \\k_2 &= hf\left(t_i + \frac{h}{2}, w_i + \frac{k_1}{2}\right) \\k_3 &= hf\left(t_i + \frac{h}{2}, w_i + \frac{k_2}{2}\right) \\k_4 &= hf(t_i + h, w_i + k_3) \\w_{i+1} &= w_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)\end{aligned}$$

Be sure to design your code to be able to handle systems of equations (e.g.  $y \in \mathbb{R}^n$ ). You may find the Matlab tutorial found at this link helpful.

*Solution.* The classical fourth-order Runge–Kutta method can be implemented directly from the formulas above. To allow the same routine to handle both a scalar differential equation and a system of differential equations, the numerical solution is treated internally as a column vector. Thus, if

$$y(t) = \begin{bmatrix} y_1(t) \\ \vdots \\ y_n(t) \end{bmatrix},$$

then  $f(t, y)$  returns an  $n \times 1$  column vector.

For a fixed number of steps  $N$ , define

$$h = \frac{b - a}{N}$$

and

$$t_i = a + ih, \quad i = 0, 1, \dots, N.$$

Starting with

$$w_0 = \alpha,$$

the RK4 approximation is computed for  $i = 0, \dots, N - 1$  using

$$\begin{aligned} k_1 &= hf(t_i, w_i), \\ k_2 &= hf\left(t_i + \frac{h}{2}, w_i + \frac{k_1}{2}\right), \\ k_3 &= hf\left(t_i + \frac{h}{2}, w_i + \frac{k_2}{2}\right), \\ k_4 &= hf(t_i + h, w_i + k_3), \end{aligned}$$

followed by

$$w_{i+1} = w_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

The following Matlab function implements this algorithm.

```
function [t,w] = rk4(f,a,b,alpha,N)
%RK4 Classical fourth-order Runge-Kutta method.
%
% [t,w] = rk4(f,a,b,alpha,N)
%
% Approximates the solution of
%
%      y' = f(t,y),      a <= t <= b,
%      y(a) = alpha,
%
% using N equally spaced RK4 steps.

h = (b-a)/N;

% Convert the initial condition to a column vector.
alpha = alpha(:);
n = length(alpha);

% Construct the time mesh.
t = linspace(a,b,N+1).';
```

```

% Preallocate storage for the numerical solution.
% Each row contains one approximation.
w = zeros(N+1,n);
w(1,:) = alpha.';

for i = 1:N

    % Current approximation as a column vector.
    wi = w(i,:).';

    k1 = h*f(t(i),wi);

    k2 = h*f(t(i) + h/2, ...
              wi + k1/2);

    k3 = h*f(t(i) + h/2, ...
              wi + k2/2);

    k4 = h*f(t(i) + h, ...
              wi + k3);

    wi_next = wi + ...
              (k1 + 2*k2 + 2*k3 + k4)/6;

    w(i+1,:) = wi_next.';
end
end

```

For example, for the scalar initial value problem

$$y' = y - t^2 + 1, \quad y(0) = 0.5,$$

on  $0 \leq t \leq 2$ , the routine can be called using

```
f = @(t,y) y - t.^2 + 1;
```

```
[t,w] = rk4(f,0,2,0.5,10);
```

```

disp ([t,w])

%=====Outputs=====
% >> gl4prob1
% 0      0.5000
% 0.2000    0.8293
% 0.4000    1.2141
% 0.6000    1.6489
% 0.8000    2.1272
% 1.0000    2.6408
% 1.2000    3.1799
% 1.4000    3.7323
% 1.6000    4.2834
% 1.8000    4.8151
% 2.0000    5.3054

```

The same routine works without modification for a system. For example, consider

$$\begin{aligned} y_1' &= y_2, & y_1(0) &= 0, & y_2(0) &= 1. \\ y_2' &= -y_1, \end{aligned}$$

Writing the system in vector form gives

$$y' = \begin{bmatrix} y_2 \\ -y_1 \end{bmatrix}.$$

The corresponding Matlab code is

```

f = @(t,y) [y(2); -y(1)];

alpha = [0;1];

[t,w] = rk4(f,0,2*pi,alpha,100);

plot(t,w(:,1),'LineWidth',1.5)
hold on
plot(t,w(:,2),'LineWidth',1.5)
legend('y_1','y_2')
xlabel('t')

```

```
ylabel('Approximate solution')
grid on
```

This code yields the plot in Figure 1 below:

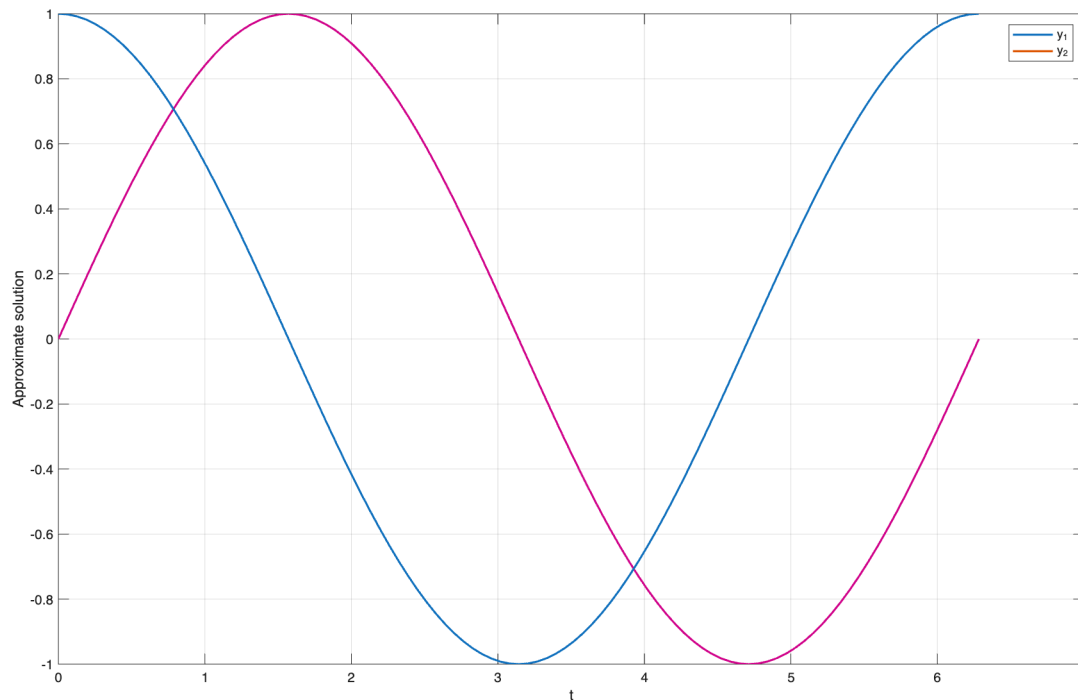


FIGURE 1. A solution for  $y_1' = y_2$  and  $y_2' = -y_1$ .

Because  $w_i$ ,  $k_1$ ,  $k_2$ ,  $k_3$ , and  $k_4$  may all be vectors, the RK4 formulas are unchanged when solving a system of equations. The classical RK4 method has global truncation error

$$O(h^4).$$

Consequently, in the asymptotic error regime, reducing the step size by a factor of two reduces the global discretization error by approximately  $2^4 = 16$ .  $\square$

**2.** Consider the initial value problem

$$(1) \quad \frac{dy}{dt} = 1 - y + e^{2t}y^2 \quad (0 \leq t \leq 0.9), \quad y(0) = 0$$

- (1) Verify that the exact solution is  $y(t) = e^{-t} \tan(e^t - 1)$ .
- (2) Use your RK4 code to approximate the solution to the given initial value problem, using a stepsize of  $h = 0.1$ . Store the approximate solution at each time step as an element in a vector  $w$ . Also add a vector to your code which stores the errors,  $err = x - w$ .
- (3) Set up a script that solves the initial value problem for different step sizes  $10^{-9} \leq h \leq 10^{-1}$  and stores the error at  $t = 0.9$ . Run this script on VCU's Athena cluster by submitting it as batch job (not interactively!). More information on how to do this can be found [here](#)). If you are using Matlab, you can find an example script for submitting your job [here](#). Make a log-log plot of the error vs. the stepsize and use this to verify the method converges with rate  $O(h^4)$ .

challenge: Assuming your code stores the solution at every step, you will likely become memory-limited if you try to test convergence using smaller step sizes. Modify your code to only save the final value of the solution, and go down to step sizes small enough to observe the effects of round-off error.

*Solution.*

□

**3.** You will now use your RK4 code to reproduce figure 7.21 in your textbook. Consider the model for cell production in a chemostat

$$\begin{aligned} \dot{s} &= 1 - s - \frac{msx}{a + s} \\ \dot{x} &= \frac{msx}{a + s} - x \end{aligned}$$

with  $m = 16$  and  $a = 0.25$ .

- (1) Find the fixed points of the system and determine their stability.
- (2) Apply RK4 with step size  $h = 0.05$  to approximate the solution to the initial value problem for  $0 \leq t \leq 12$  with initial conditions  $s(0) = 0.5$  and  $x(0) = 0.02$ . Plot  $s(t)$  and  $x(t)$  and explain how your results are consistent to your linear stability analysis.
- (3) Now repeat the previous part, this time using  $h = 0.06$  to approximate the solution. Comment on the differences in your plots for  $s(t)$  and  $x(t)$ .

*Solution.*

□

**4.** In order to explore absolute stability of RK4, consider the test problem  $y' = \lambda y$  with  $y(0) = 1$  and  $0 \leq t \leq 10$ .

- (1) Analytically apply RK4 to the test problem to show that

$$w_{i+1} = \left( 1 + h\lambda + \frac{1}{2}(h\lambda)^2 + \frac{1}{6}(h\lambda)^3 + \frac{1}{24}(h\lambda)^4 \right) w_i.$$

This allows us to define  $Q(h\lambda) = 1 + h\lambda + \frac{1}{2}(h\lambda)^2 + \frac{1}{6}(h\lambda)^3 + \frac{1}{24}(h\lambda)^4$ .

- (2) Now we will determine the stability region. Allow  $h\lambda$  be a complex number  $h\lambda = x+iy$  where  $i = \sqrt{-1}$ . Create a contour plot that shows the  $|Q| = 1$  level set over the complex plane. If you are using Matlab, you may find the command `meshgrid` useful for creating a grid of points in the complex plane as follows:

MATH 515

Numerical Analysis

Spring 2026

```
% Define the grid for the complex plane (z = h*lambda)
x = linspace(-3, 3, 301);
y = linspace(-3, 3, 301);
[X, Y] = meshgrid(x, y);
Z = X + 1i*Y;
```

The Matlab command `contour()` can then be used to create the requested plot. Determine which region in the plot is the stability region.

- (3) With  $\lambda = -55.31$  determine a bound on  $h$  that guarantees  $h\lambda$  will lie in the stability region for this method. Numerically apply RK4 to the test problem for  $h$  values inside and outside of this stability region. How do the solution differ? How do these results inform the differences you noticed in parts 3(b) and 3(c)?
- (4) We will see what happens to numerical solutions with  $h\lambda$  combinations that lie in both the region of stability and the right half of the complex plane,  $\text{Re}(\lambda h) > 0$ . To explore this, set  $\lambda = 1 + 20i$  and  $h = 0.1$ . The point  $h\lambda$  should now lie in both the right half of the plane and in the region of stability – plot this point and the region of stability on the same axes.

The solution with this  $\lambda$  is

$$e^{(1+20i)t} = e^t(\cos(20t) + i\sin(20t)).$$

Should the solution converge to zero as  $t \rightarrow \infty$ ? Apply the RK4 to the test problem with these values of  $\lambda$  and  $h$ . Does the numerical method converge to zero? Should you use a  $h\lambda$  that falls in or outside of the stability region for this  $\lambda$ ?

*Solution.*

□